

Clone Hero Star Power Optimizer

Jason Montas, Sidney Hymes
March 17, 2026



Table of Contents

- ① README
- ② Why is an Algorithm Needed?
- ③ Project Structure
- ④ Development Focus
- ⑤ Working Example

README Overview (Intro/Goals)

Our goal for this project is to create an algorithm that finds the optimal star power path for any chart in the game Clone Hero to achieve a high score. We will create a website that serves as a GUI for this algorithm.

README Overview (What is Clone Hero?)

Clone Hero is a rhythm game based on the popular Guitar Hero series. The gameplay is as follows: notes will appear on what is referred to as a highway, and you hit the notes by pressing/strumming the corresponding buttons using a plastic guitar controller while a song plays.

Clone Hero is a community driven game, with people creating charts for the game, while Guitar Hero works like any other game. You buy the game, and you have a limited number of songs that come with the game.

Highway and Controller



(a) Highway with notes



(b) Plastic Guitar Controller

README Overview (How Scoring in Clone Hero Works)

In Clone Hero there are two types of notes. Standard notes give 50 points when hit. Sustain notes are similar to standard notes, but holding down the note gives more points per bar held. As you hit more notes without missing, your score multiplier meter will increase, with a max of a 4x multiplier.

How Scoring Works cont.

In the game there is something called star power, which doubles the score multiplier when activated. In order to obtain star power, there are sections called star power phrases. When you hit all notes in the star power phrase, the meter will increase by 25%. When the meter hits 50%, you can then activate star power. Generally speaking, the best time to activate star power is when there is a section packed with notes, so that you can maximize the usefulness of it.

README Overview (How the Algorithm Functions)

The algorithm currently works as a command-line pipeline. The user gives the program an input chart file and an output image path. The input can be either a .chart file or a .mid file.

First, the parser reads the file and converts it into a shared internal chart model. This model contains note timing, sustain lengths, star power phrase ranges, tempo events, time signatures, and chart resolution.

How the Algorithm Functions cont.

Next, the optimizer evaluates possible star power activation decisions across the song and computes the activation path that gives the highest total score under the implemented game rules. Finally, the renderer generates a PNG image of the full chart and highlights the recommended activation points so the result is easy to follow visually.

README Overview (Tools)

We are using Java for the majority of this project. We will use HTML, CSS, and JavaScript to create a website that will serve as a GUI for input and output.

Table of Contents

- ① README
- ② Why is an Algorithm Needed?
- ③ Project Structure
- ④ Development Focus
- ⑤ Working Example

Why is an Algorithm Needed?

While achieving a high score may seem easily achievable (waiting to use star power during the densest sections of a song), it is actually a little more complicated. With evidence from this project alone, there are many variables that can slightly alter your score. This is why an algorithm is needed to efficiently view the entire chart at once and plan accordingly.

Table of Contents

- ① README
- ② Why is an Algorithm Needed?
- ③ Project Structure
- ④ Development Focus
- ⑤ Working Example

There are 3 major components of my project which are the following:

1. Chart Parser
2. Optimization Algorithm
3. Image/Chart Renderer

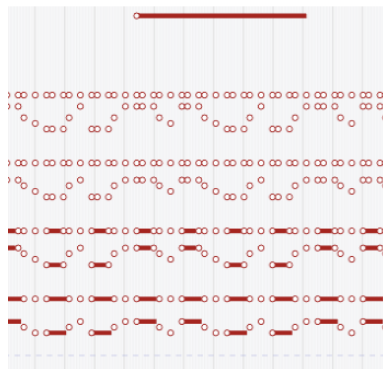
How it works

- Its first job is to read the input file (.chart/.mid) and normalize it into one structure called ChartData. That structure stores:
 - notes
 - star power phrases
 - tempo changes
 - time signatures
 - chart resolution
- This is a very important part of the process since it converts 2 vastly different file types into a format that the algorithm can read and understand.

.chart vs .mid

```
10080 = N 1 0
10200 = N 0 0
10200 = N 1 0
10320 = N 0 0
10320 = N 1 0
10440 = N 0 0
10440 = N 1 0
10560 = N 2 0
10560 = N 5 0
10560 = S 2 1080
10680 = N 3 0
10800 = N 2 0
10920 = N 3 0
11040 = N 2 180
11280 = N 7 180
11520 = N 4 0
11640 = N 7 0
11640 = N 5 0
11760 = N 4 0
11880 = N 3 0
```

(a) .chart format



(b) .mid format

Optimization Algorithm

- We used dynamic programming to structure the algorithm since it best fits the major question when activating star power: Is it best to activate now or later?
- How the algorithm works:
 - Sorts notes by time
 - Groups notes that appear at the same time into one scoring unit
 - Computes the base scoring characteristics of each group, including sustains and combo multiplier
 - Locates where star power phrases are completed
 - Explores two choices at each step: activate now, or do not activate now
 - Stores solved states in memory

Dynamic Programming State Structure

- Note group
- Percentage of star power meter
- State of star power (active or not)

The process ends by rendering an image of the entire chart along with highlighted star power phrase sections and activation points.

Table of Contents

- ① README
- ② Why is an Algorithm Needed?
- ③ Project Structure
- ④ Development Focus
- ⑤ Working Example

Chart Parser

The most challenging part of the project. Getting this down was integral, and it took awhile to figure out how to format everything correctly and include both file types.

Development Focus

Chart Parser

The most challenging part of the project. Getting this down was integral, and it took awhile to figure out how to format everything correctly and include both file types.

Optimization Algorithm

The hardest part was making sure everything was accurate to how score is actually computed in the game.

Development Focus

Chart Parser

The most challenging part of the project. Getting this down was integral, and it took awhile to figure out how to format everything correctly and include both file types.

Optimization Algorithm

The hardest part was making sure everything was accurate to how score is actually computed in the game.

Image Renderer

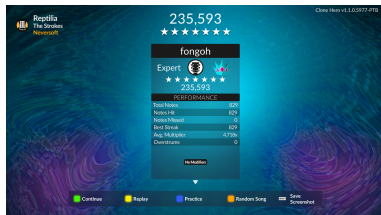
This was the part I had most fun with.

Table of Contents

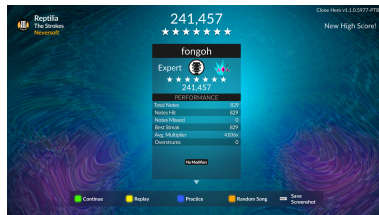
- ① README
- ② Why is an Algorithm Needed?
- ③ Project Structure
- ④ Development Focus
- ⑤ Working Example

Example for Reptilia by The Strokes

Score Comparison

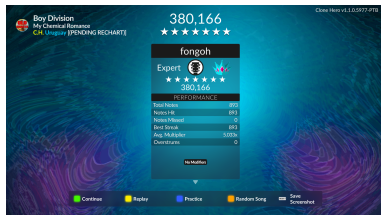


(a) Before Algorithm

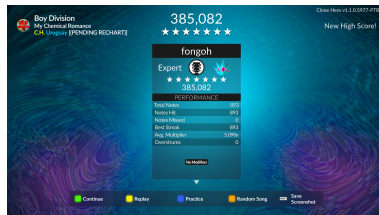


(b) After Algorithm

Score Comparison

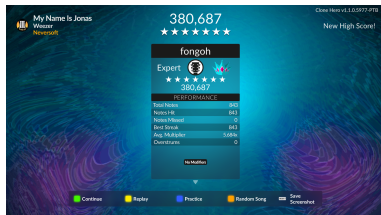


(a) Before Algorithm

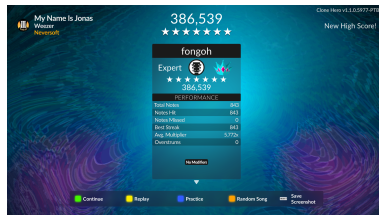


(b) After Algorithm

Score Comparison



(a) Before Algorithm



(b) After Algorithm

- Tweak algorithm even further
- Make a GUI for project
- Add a feature that notifies player when to activate star power in real time

Thank You!
Questions?